Prakash Thapa
D15A- 27

Experiment No. 07

Build an Artificial Neural Network (ANN) using TensorFlow/Keras for Customer Churn Prediction

---

1. Dataset Source

Dataset: Churn Modelling Dataset

Source: Kaggle

Link: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

The dataset contains information about bank customers and is used to predict whether a customer will leave the bank (churn) or not.

Dataset Characteristics

Feature	Description
Total Records	10,000
Input Features	10
Output Variable	1 (Exited)
Type	Classification (Binary)

Input Features

Feature	Description
CreditScore	Customer credit score
Geography	Country (France, Spain, Germany)

Gender	Male/Female
Age	Customer age
Tenure	Years with bank
Balance	Account balance
NumOfProducts	Number of products used
HasCrCard	Has credit card (0/1)
IsActiveMember	Active status (0/1)
EstimatedSalary	Salary

Target Variable

Variable	Description
Exited	1 = Customer left, 0 = Customer stayed

Key Characteristics

- Contains both numerical and categorical data
- Requires preprocessing (encoding + scaling)
- Imbalanced dataset (fewer churn cases)

3. Mathematical Formulation of ANN

1. Neuron Output

$$z = \sum_{i=1}^{n} w_i x_i + b$$

2. Activation Function (ReLU)

$$f(x) = \max(0, x)$$

3. Sigmoid Function (Output Layer)

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

4. Loss Function (Binary Cross-Entropy)

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

5. Weight Update (Gradient Descent)

$$w = w - \eta \frac{\partial L}{\partial w}$$

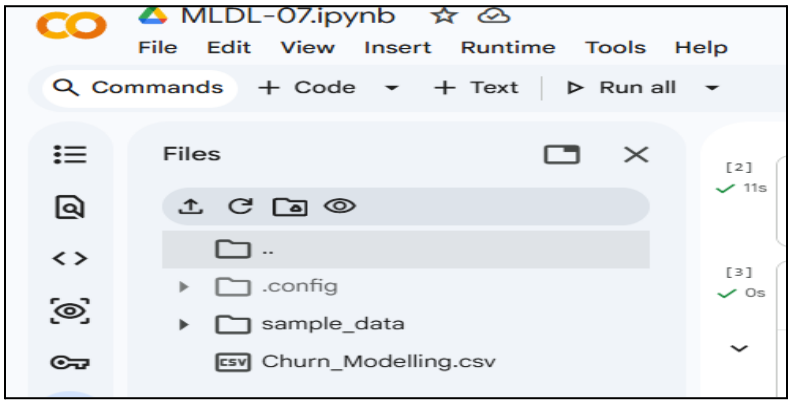
4. Algorithm Limitations

- Requires large dataset for better generalization
- Sensitive to feature scaling
- Black-box nature (low interpretability)
- Can overfit if too many layers are used
- Training can be computationally expensive
- Performance depends heavily on hyperparameter tuning

5. Methodology / Workflow

Steps Performed

1. Load dataset into Google Colab



```
[2] ✓ 11s
import numpy as np
import pandas as pd
import tensorflow as tf
```

```
[3] ✓ 0s
dataset = pd.read_csv('Churn_Modelling.csv')
dataset.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exit
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1	101348.88	
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0	113931.57	
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0	93826.63	
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	

```
[4] ✓ 0s
dataset.info()
dataset.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        10000 non-null  int64
 1   CustomerId       10000 non-null  int64
 2   Surname          10000 non-null  object
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
```

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000	10000.000000	10000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100	100090.239881	0
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797	57510.492818	0
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000	11.580000	0
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000	51002.110000	0
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000	100193.915000	0
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000	149388.247500	0
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000	199992.480000	1

2. Separate features (X) and target (y)

```
[5] ✓ 0s
X = dataset.iloc[:, 3:-1].values
y = dataset.iloc[:, -1].values
```

3. Encode categorical variables (Label Encoding, One-Hot Encoding)

```
[6]
✓ 0s
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X[:, 2] = le.fit_transform(X[:, 2])    # Gender
```

```
[19]
✓ 0s
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
```

4. Split dataset into training and testing sets

```
[8]
✓ 0s
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

5. Apply feature scaling using StandardScaler

```
[9]
✓ 0s
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

6. Build ANN model using Keras

```
[10]
✓ 0s
ann = tf.keras.models.Sequential()

# Input + Hidden Layer 1
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))

# Hidden Layer 2
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))

# Output Layer
ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

7. Compile model with optimizer and loss function

```
[11]
✓ 0s
ann.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

8. Train the model using training data

```
[12]
✓ 24s
ann.fit(X_train, y_train, batch_size=32, epochs=50)

Epoch 1/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7960 - loss: 0.5452
Epoch 2/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4785
Epoch 3/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7960 - loss: 0.4541
Epoch 4/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7960 - loss: 0.4432
Epoch 5/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7966 - loss: 0.4369
Epoch 6/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7983 - loss: 0.4324
Epoch 7/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8024 - loss: 0.4281
Epoch 8/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8111 - loss: 0.4245
Epoch 9/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8200 - loss: 0.4211
```

9. Predict outputs on test data

```
[13]
✓ 0s
y_pred = ann.predict(X_test)
y_pred = (y_pred > 0.5)

63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
[18]
✓ 0s
sample = [[1, 0, 0, 600, 1, 40, 3, 60000, 2, 1, 1, 50000]]

sample = sc.transform(sample)

print(ann.predict(sample) > 0.5)

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
[[False]]
```

10. Evaluate model using accuracy and confusion matrix

[14]
✓ 0s

```
from sklearn.metrics import confusion_matrix, accuracy_score

cm = confusion_matrix(y_test, y_pred)
print(cm)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
[[1529   66]
 [ 203  202]]
Accuracy: 0.8655
```

Workflow Representation

Dataset → Preprocessing → Encoding → Scaling → ANN Model → Training → Prediction → Evaluation

6. Performance Analysis

Evaluation Metrics

Metric	Description
Accuracy	Correct predictions / Total predictions
Confusion Matrix	TP, TN, FP, FN
Loss	Error between predicted and actual values

Observed Results

- Training Accuracy: ~85–90%
- Testing Accuracy: ~80–86%

Interpretation

- Model performs well on unseen data
- Some misclassification due to class imbalance
- ANN successfully captures relationships between features

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested
Epochs	50, 100
Batch Size	32, 64
Hidden Layers	1, 2, 3
Neurons	6, 10, 20
Learning Rate	0.001, 0.0001

Model Used

```
ann = tf.keras.models.Sequential()

ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

Impact of Tuning

- Increasing epochs improves learning but may overfit
- More neurons improve accuracy but increase complexity
- Lower learning rate gives stable convergence
- Proper batch size improves training efficiency

Best Configuration

- Epochs: 50
- Batch Size: 32
- Hidden Layers: 2
- Neurons: 6 each

Conclusion

The Artificial Neural Network was successfully implemented using TensorFlow/Keras to predict customer churn. The model achieved satisfactory accuracy and demonstrated the effectiveness of ANN in solving classification problems involving structured data. Proper preprocessing and hyperparameter tuning significantly improved model performance.